

Trainer Teaching Guide: Spring Boot Session 1

- Foundations

A detailed, textbook-style script for [springboot-session1-theory.html](#). Simple English, examples, analogies and natural fillers - read it aloud and it sounds like teaching, not reading.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session1-theory.html](#) (33 slides)

HOW TO USE THIS GUIDE:

- ON SCREEN tells you what the slide shows.
- SAY is the full teaching script - read it word-for-word; fillers and questions are built in.
Pause where you see '...'.
- EMPHASISE / DELIVERY TIP are notes for you - do NOT read these aloud.

Slide 1 - Title: Spring Boot

ON SCREEN

The blue cover slide: 'Spring Boot', with the line 'from zero to a running REST API'.

SAY (read this aloud)

Good morning everyone, and a warm welcome to the very first session of our Spring Boot series. Now, some of you may have heard that Spring is big and complicated and scary. Let me put you at ease right away - Spring Boot was actually invented to make Spring easy. So you are starting at the friendly end.

Here is what Spring Boot is, in one line: it is the framework that powers most modern Java backends - the engines behind websites, mobile apps, and online services. Almost every Java company uses it.

And here is our promise for today. By the end of this session, you will go from absolutely nothing to a real, running web service that you built yourself. We will answer three simple questions along the way: why does Spring Boot exist, how does it work behind the scenes, and how do we write our first piece of it. That's it. Let's begin.

DELIVERY TIP

Keep the energy light and encouraging. Many freshers are nervous about 'frameworks'. Repeat the reassurance: Boot exists to make things easy.

Slide 2 - Agenda

ON SCREEN

Seven numbered agenda items, from why Spring Boot exists through to the hands-on labs.

SAY (read this aloud)

Let me show you the journey for today, so you always know where we are. Don't worry about the terms yet - just get a feel for the path.

We start with why Spring Boot exists - and to understand that, we will first look at how painful life was before it. Then what Spring Boot actually is, and how it works its magic. Then we will draw the architecture - how a request travels through your app - and the idea of layers. After that, we unpack the one special annotation that starts everything. Then a core idea called dependency injection, which sounds fancy but is really simple. Then we look at the project itself - the tools, the folders, the files. And finally, the hands-on lab where you build your first app.

So, theory first to build understanding, then your hands on the keyboard. Sound like a plan? Great.

Slide 3 - Life before Spring Boot

ON SCREEN

A bulleted list of the manual setup work classic Spring required, and a warning box about the real cost.

SAY (read this aloud)

To really appreciate Spring Boot, you have to feel the pain it removed. So let me take you back in time. The Spring framework came out in 2003, and it was genuinely brilliant - it gave us clean, well-organised code. But there was a catch: just getting a project to start was slow and frustrating.

Picture this. Before you could write a single line of useful code, you had to do all of this by hand. You wrote pages and pages of XML configuration. You manually connected every object to every other object. You separately downloaded and installed a web server called Tomcat, configured it, and deployed your app into it. And you hand-picked the version of every library and prayed they all worked together.

Honestly, teams would spend an entire day just to get an empty project running. A whole day, before any real work. That is the frustration Spring Boot was built, in 2014, to wipe away. So as we go through today, every feature of Boot is really an answer to one of these old pains. Keep that in mind.

Slide 4 - XML configuration hell

ON SCREEN

A block of classic Spring XML defining beans, plus bullet points on why it was painful.

SAY (read this aloud)

Let me show you pain point number one, up close. In the old Spring, the framework could not find your objects by itself - you had to describe every single one in an XML file. Look at the screen - this is what it looked like. Each object is declared by hand, and you manually wire one to another with these ref tags.

Now imagine three problems with this. First, every new class you write means another chunk of XML to maintain. Second - and this one really hurt - if you made a typo in a class name here, the compiler would not catch it. Your code looked fine, it compiled fine, and then it crashed when you ran it. The error was hidden until the worst moment. And third, on real projects, this XML grew into hundreds, sometimes thousands of lines. People joked that they spent more time writing XML than actual Java.

So you can see why developers were tired. Spring Boot's answer is to throw nearly all of this away and replace it with a couple of simple annotations that the framework discovers automatically. We will see exactly how, very soon.

Slide 5 - Wiring, servers & version juggling

ON SCREEN

Four cards: manual wiring, no embedded server, version conflicts, repeated boilerplate.

SAY (read this aloud)

Here are pain points two, three, and four, shown as four cards. Let me go through them quickly, because together they reveal something important.

Manual wiring - you connected every dependency yourself, and if you forgot one, the app crashed at runtime, not before. No built-in server - you had to build a special file, install Tomcat, configure it, and deploy into it; four slow manual steps. Version conflicts - you matched the versions of Spring, the JSON library, the logging library, all by hand, and one wrong combination caused the famous 'dependency hell'. And repeated boilerplate - the same setup code copied into every new project before you could even begin.

Now, here is what I want you to notice about all four. Every single one is the same kind of work for almost every project. It is boring, repetitive setup that does not change much from app to app.

EMPHASISE

Land the key insight on the slide: if ninety-five percent of projects set these up the same way, why should every developer do it by hand? The framework should just do it by default, and let you change only the rare bits that differ. That idea is called 'convention over configuration', and it is the soul of Spring Boot. Say it clearly - it ties the whole session together.

Part 2 - What Spring Boot Is

Slide 6 - What is Spring Boot?

ON SCREEN

A vertical stack: Your Application runs on Spring Boot, which is built on the Spring Framework. A 'mental model' key box.

SAY (read this aloud)

So here is the solution to all that pain. Spring Boot is a layer that sits on top of the regular Spring framework and makes it easy to create applications that you can just run, straight away, with almost no setup.

Now there is one thing I really want to clear up, because people get confused. Spring Boot does NOT replace Spring. Please remember that. You still write the same Spring code - the same controllers, the same services. Boot just removes all the annoying setup that used to surround them. Look at the stack on screen - your app sits on top, running on Spring Boot, which is built on the Spring framework underneath. They work together.

EMPHASISE

Give them the mental model from the key box - it really helps: Spring is the engine of a car; Spring Boot is the self-starting ignition and the auto-tuning. The engine was always powerful, but now it starts instantly and tunes itself. So everything they learn about core Spring still applies - Boot just means less time on setup, more time on features.

Slide 7 - The four things Boot provides

ON SCREEN

Four cards: auto-configuration, embedded server, starter dependencies, production-ready.

SAY (read this aloud)

Everything Spring Boot does fits neatly into four big gifts. If you understand these four, you understand Spring Boot itself. Let me introduce each one - we will go deeper on all of them in the next few slides.

Gift one - auto-configuration. Boot looks at what is in your project and configures sensible defaults automatically. This is what replaced all that XML. Gift two - an embedded server. The Tomcat web server is now built right inside your app, so there is nothing external to install. You run your app like an ordinary Java program. Gift three - starter dependencies. Instead of hunting down a dozen libraries, you add one 'starter' and it brings in everything that belongs together, with versions guaranteed to fit. And gift four - production-ready features. Things like health checks and metrics, built in, so your app is ready for the real world.

So, four gifts: auto-config, embedded server, starters, and production features. Together, they turn that whole painful day of setup into simply 'run the main method'.

Slide 8 - Convention over configuration

ON SCREEN

Bullet examples of Boot making intelligent guesses from the classpath, and a key box on day-to-day impact.

SAY (read this aloud)

Let's spend a moment on the philosophy that makes all of this possible - convention over configuration. It sounds academic, but the idea is simple and clever.

Instead of forcing you to spell out every little detail, Spring Boot assumes the most common, sensible setup - that is the convention - and it only asks you to specify something when your needs are different - that is the configuration. In other words, it makes smart guesses.

Here are some real guesses it makes. It sees you have the web library, so it assumes you are building a web app and starts Tomcat on port 8080 - no questions asked. It sees a database driver, so it assumes you want a database and sets up the connection. It finds a class marked as a controller, so it automatically wires it up to handle web requests.

So what does this mean for you day to day? It means you write configuration only when you want something unusual. If the defaults suit you - and most of the time they do - you write nothing at all. Think of it like ordering at a restaurant where the chef already knows the perfect default meal; you only speak up if you want to change something.

Slide 9 - Auto-configuration, step by step

ON SCREEN

A three-box flow: scan classpath, check conditions, configure beans. Bullets on conditional activation.

SAY (read this aloud)

Now let me peek under the hood at how auto-configuration actually works, because it is the mechanism that replaced all that XML, and it is genuinely clever.

When your app starts, Spring Boot runs through hundreds of little pre-written configuration recipes. And each recipe asks itself a yes-or-no question: 'given this particular project, should I switch on?' Follow the flow on screen. First, it scans your project to see which libraries are present. Then it checks conditions - little rules like 'only activate if this library is here, and only if the developer has not already done this themselves'. Then it configures just the things that are actually needed.

So the key word is conditional - each piece of setup only happens if it is relevant to your project. That is exactly why simply adding a starter is enough to switch on a whole feature. And here is a thoughtful touch: if you decide to configure something yourself, Boot notices and politely steps back, using yours instead of its default. It never fights you.

DELIVERY TIP

Share the practical debugging trick on the slide: running the app with `dash-dash-debug` prints a full report of what auto-configured and why. Tell them it is a lifesaver when something is not set up the way they expect.

Slide 10 - The embedded server

ON SCREEN

Bullets on the fat JAR, plus a before/after comparison of traditional WAR deployment versus Spring Boot.

SAY (read this aloud)

Let's talk about the embedded server, because this was a genuine game-changer. In the old days, your app and the web server were separate. You built your app into a file, then you installed a Tomcat server on the machine, configured it, and dropped your app inside it. Multiple slow steps, and lots of room for mistakes.

Spring Boot flips this completely. The web server now lives inside your application. So running your main method starts both your app and its own web server, in one go. Everything packages into a single file - we call it a 'fat JAR' because it contains everything: your code, all the libraries, and the server itself. To deploy it anywhere, you just copy that one file to any machine that has Java, and type `'java dash jar'` and the file name. Done.

Look at the before-and-after on screen - four manual steps on the left collapse into 'build one file, run it' on the right. Think of it like the difference between buying furniture you have to assemble versus furniture that arrives ready to use. Boot apps arrive ready to run.

Slide 11 - Starter dependencies

ON SCREEN

The starter-web XML snippet, then a flow showing one line pulling in MVC, Jackson, Validation, and Tomcat.

SAY (read this aloud)

Now, starter dependencies - another lovely time-saver. A 'starter' is a ready-made bundle of libraries for one job. Instead of researching and downloading ten separate libraries and matching their versions, you add one starter, and it pulls in everything that belongs together.

Look at the example - that single dependency, spring-boot-starter-web, quietly brings in Spring's web tools, the Jackson library for JSON, validation, and the embedded Tomcat. One line, four things, all guaranteed to work together.

Let me give you an analogy. It is like buying a 'breakfast combo' at a cafe instead of ordering the eggs, the toast, the juice, and the coffee separately and hoping they arrive together. The combo is curated for you. And here is the part that ends dependency hell: the Spring team has already tested all the versions inside a starter to make sure they get along. You do not even pick versions - they are handled for you. Upgrade Boot once, and everything moves up together, safely.

Slide 12 - Common Spring Boot starters

ON SCREEN

A table of the starters they'll meet most: web, data-jpa, security, test, thymeleaf, validation.

SAY (read this aloud)

There are dozens of official starters, and they all follow the same naming pattern - spring-boot-starter-something. Let me quickly point out the ones you will meet most often, so the names feel familiar later.

starter-web - for web apps and REST APIs; that is the one we use today. starter-data-jpa - for talking to databases, which we add in Session 2. starter-security - for logins and protecting your app. starter-test - which brings all the testing tools, and it is actually added for you automatically. starter-thymeleaf - for building HTML web pages. And starter-validation - for checking that incoming data is valid.

Do not memorise these. Just absorb the pattern: as your app grows and needs a new capability, there is almost certainly a starter for it. Need a database next session? Add the JPA starter. That is the rhythm. Today, we keep it simple and use just the web starter.

Slide 13 - Production-ready features

ON SCREEN

Bullets on Actuator endpoints - health, metrics, info - and a note that it's covered later.

SAY (read this aloud)

The fourth gift is production-ready features. Here is the thing - building an app is only half the job. Once it is live and serving real users, you need to keep an eye on it. Is it healthy? How much memory is it using? How many requests is it handling right now? Spring Boot gives you all of this through a module called Actuator, with almost no effort.

It adds special web addresses you can visit. One called slash-health tells you if the app and its database are healthy - and big systems use this to decide whether to send traffic to your app. One called slash-metrics shows memory, CPU, and request timings. One called slash-info shows version details. These feed into monitoring dashboards that operations teams watch all day.

We will not set this up today - that comes later. The point I want you to take away right now is just the idea: with Spring Boot, these professional, production concerns are built in from the start, not bolted on at the end in a panic. That is what the word 'production-grade' really means.

Part 3 - Architecture & How It Starts

Slide 14 - How a request flows

ON SCREEN

A horizontal architecture diagram: Browser, HTTP Request, DispatcherServlet, Controller, Service, Repository, Database.

SAY (read this aloud)

This is the most important picture of the entire day, so let's slow down and trace a request all the way through, like following a parcel through the postal system.

A browser sends a request - say, GET slash hello. It arrives at a special gatekeeper called the DispatcherServlet. Now, this is Spring's 'front controller' - think of it as the front desk of a big office. Every visitor, every request, comes to this one desk first. The front desk looks at the address - slash hello - and routes the request to the correct controller method. The controller then asks a service to do the actual thinking - the business logic. The service asks a repository to fetch or save data. And the repository talks to the database. Then the answer travels all the way back along the same path to the browser.

So the chain is: browser, front desk, controller, service, repository, database - and back. Notice each box has exactly one clear job.

EMPHASISE

Name the principle - this is the big takeaway: each component doing one job is called separation of concerns. It is what keeps apps easy to test and easy to change. Tell them to keep this diagram in mind - we revisit it through the whole series.

Slide 15 - Responsibilities by layer

ON SCREEN

A stacked diagram of four layers: Controller, Service, Repository, Database, each with its responsibility.

SAY (read this aloud)

Let me zoom in on those layers and give each one a clear identity, because we will use these words constantly.

The Controller is the receptionist. It takes the web request, reads the input, calls a service, and hands back the response. It knows about the web, but it does not contain the actual business rules. The Service is the brain. This is where the real thinking happens - calculations, decisions, validations. But the brain knows nothing about the web or about databases - it just thinks. The Repository is the librarian. Its only job is fetching and storing data - find, save, update, delete. And the Database is the storeroom, where everything is kept safely.

Why split it up like this? Here is the payoff. Because each layer only talks to the one right below it, you can change one without breaking the others. You could swap the database, or redesign the web part, or test the brain on its own - and the boundaries protect everything else. Today's lab builds the controller and the service; the repository and database arrive in Session 2.

Slide 16 - @SpringBootApplication

ON SCREEN

The main class with `@SpringBootApplication` and the `SpringApplication.run` line, plus four bullets of what `run()` does.

SAY (read this aloud)

Every Spring Boot app has exactly one special main class, marked with this annotation - `@SpringBootApplication`. And running its plain, ordinary Java main method is the thing that brings your whole application to life.

Look at the code - it is tiny. There is really just one line that matters: `SpringApplication.run`. But do not let its small size fool you - that one line does an enormous amount of work behind the scenes. Let me tell you what it does, in order. It creates the container that will hold all your objects. It runs the auto-configuration we talked about, based on your libraries. It scans your project and finds all your components - controllers, services - and registers them. And it starts the embedded Tomcat server.

So by the time that one line finishes running, your web app is live and listening on port 8080. One annotation, one line - and an entire application springs to life. Pretty elegant, isn't it?

Slide 17 - @SpringBootApplication, unpacked

ON SCREEN

Three rows mapping the bundled annotations: Configuration, EnableAutoConfiguration, ComponentScan. A warning about package placement.

SAY (read this aloud)

That annotation looks like magic, but let me take the mystery away. `@SpringBootApplication` is really just a convenience - a single sticker that secretly combines three smaller stickers. Knowing the three makes everything clear.

The first is `@Configuration` - it says 'this class can define objects for Spring'. The second is `@EnableAutoConfiguration` - this is the one that switches on all that automatic setup based on your libraries. And the third is `@ComponentScan` - this one tells Spring to go searching through this class's package, and everything below it, to find your controllers and services and register them.

Now, that third one leads to the single most common beginner bug, so let me warn you about it now.

EMPHASISE

Stress the package rule hard - it saves hours of confusion. Because component scanning starts at the main class's package and only searches downward, your controllers and services **MUST** live in that package or a sub-package of it. If you put a controller somewhere else, Spring never finds it, and your endpoint silently returns a 404 with no error message. When something 'just does not work', this is the first thing to check.

Part 4 - Dependency Injection & IoC

Slide 18 - Dependency Injection: the why

ON SCREEN

Bullets on the benefits of DI - looser coupling, easier testing, less plumbing, single responsibility - and a one-line definition.

SAY (read this aloud)

Now a core idea that sounds intimidating but is actually simple - dependency injection. Let me build it up gently.

Almost every object needs other objects to do its job. A controller needs a service; a service needs a repository. The question is: who creates those helpers? The old way was that an object created its own helpers, using the 'new' keyword. Dependency injection says: no, an object should not build its own helpers. Instead, someone else builds them and hands them in, ready to use.

Why is that better? A few reasons. Your class becomes flexible - it depends on what it needs, not on how that thing is built. Testing becomes easy - you can hand in a fake helper during a test. You write far less wiring code, because Spring builds and supplies the helpers. And each class can just

focus on its own job.

EMPHASISE

Give them the famous one-liner - it makes the idea click: 'Don't call us, we'll call you.' Your class simply declares what it needs, and the framework provides it. That flip in who is in charge is exactly what 'Inversion of Control' means. Use a real-world image: it is like a hotel where you do not cook your own breakfast - you just say what you want and the kitchen brings it.

Slide 19 - Tight vs loose coupling

ON SCREEN

Two code panels: tight coupling using 'new', versus loose coupling using @Autowired. A real-world payoff note.

SAY (read this aloud)

Let me make dependency injection concrete by comparing two pieces of code. The fancy terms are 'tight coupling' and 'loose coupling', but the idea is everyday.

On the left, tight coupling. The controller writes 'new GreetingService' itself. So now it is permanently glued to that one exact class. If you ever want to swap in a different version, or a fake one for testing, you cannot - not without editing the controller's own code. It is like a lamp with the bulb soldered in; if the bulb dies, you throw away the whole lamp.

On the right, loose coupling. The service is injected with at-Autowired. The controller just says 'I need a greeting service' and Spring decides what to hand over. Now you can swap it, wrap it, or fake it freely, and the controller never changes. It is like a lamp with a normal socket - you change the bulb whenever you like.

The real-world payoff is on the slide. Suppose later you need to log every greeting, or fetch greetings from a database. With loose coupling, you just supply a different helper - the controller does not change at all. That flexibility is what keeps big codebases sane.

Slide 20 - Components, stereotypes & @Autowired

ON SCREEN

Bullets on @Component and @Autowired, then a table of stereotype annotations: RestController, Service, Repository, Component.

SAY (read this aloud)

So, for Spring to hand you a helper, it must first know that helper exists. You tell Spring by putting little labels - annotations - on your classes. There are two halves to this: registering a helper, and asking for one.

at-Component is how you register one. It tells Spring, 'please create and manage an instance of this class for me'. Once Spring creates it, we call it a bean - that is just Spring's word for an object it manages. And at-Autowired is how you ask for one - it says 'Spring, inject a matching bean right here, please'.

Now look at the table. These - at-RestController, at-Service, at-Repository - are all really just

specialised versions of `@Component`. They register a bean exactly the same way. So why have different names? Because each one also signals the class's role, which makes your code self-explanatory. `@RestController` for the web layer, `@Service` for the brain layer, `@Repository` for the data layer. Same registration, clearer meaning. It is like name tags at a conference that also say your job title.

Slide 21 - The IoC container

ON SCREEN

A dashed 'Spring IoC Container' box holding three beans, with a note that Spring injects them where `@Autowired` asks.

SAY (read this aloud)

Let's give a name to the thing doing all this work. The principle is Inversion of Control, and the part of Spring that actually carries it out is called the IoC container - also known as the application context. Think of it as a smart manager that runs the whole show.

When your app starts, this manager scans for all your labelled classes, creates one instance of each - by default just one, shared everywhere - figures out what each one needs, and injects those needs automatically. Look at the diagram - the container is holding your controller, your service, and your repository, all created and wired by it.

So in our little app, the container creates the greeting service once, notices that the controller needs a greeting service, and hands the same one over. And notice - you never wrote 'new' anywhere. That is the 'inversion': normally your code is in charge of creating things, but here the framework takes charge and assembles your objects for you. You just declare what you need and let the manager handle the rest.

Slide 22 - Field, constructor & setter injection

ON SCREEN

Two code panels - constructor injection (recommended) and field injection (simplest) - plus bullets on the three styles.

SAY (read this aloud)

There are three places Spring can inject a helper, and it is worth knowing the trade-offs, because professionals have a clear preference.

The recommended one, on the left, is constructor injection. The helper is passed in when the object is built, and stored in a field marked 'final', meaning it can never change. The benefits are real: the helper can never be missing or null, the class is easy to create in tests, and it is obvious what the class depends on, just by looking at its constructor. This is the modern best practice.

The simplest one, on the right, is field injection - you just put `@Autowired` right on the field. It is the shortest to write, which is why we use it in today's lab for clarity. But honestly, it is harder to test and it hides what the class really needs, so in real projects you should prefer the constructor.

And there is a third, setter injection, where Spring calls a setter method - that one is best for optional helpers that might change later. So: constructor for required helpers, setter for optional

ones, field for quick learning examples.

DELIVERY TIP

Be honest with them: 'In today's lab we use field injection because it is the simplest to read while learning. But in your real job, prefer constructor injection.' Setting that expectation now saves confusion later.

Part 5 - REST & The Project

Slide 23 - What is REST?

ON SCREEN

Bullets defining resources, verbs, statelessness, and JSON; a small flow of a GET request and response.

SAY (read this aloud)

Let's talk about the style our web service follows - it is called REST. The word is an abbreviation, but forget the long form; here is the simple idea. You model your application as a set of 'resources' - which are just nouns, things - and each thing has its own web address. Then you act on those things using standard verbs.

So a resource is a thing - a user, an order, a product, a greeting. It lives at an address like slash-users-slash-42. And you act on it with an HTTP verb: GET to read it, POST to create one, PUT to update it, DELETE to remove it. Nouns for the things, verbs for the actions. Very natural, like a language.

One more important property - REST is stateless. That means each request carries everything the server needs to answer it, and the server keeps no memory of you between requests. Think of it like ordering at a counter where they do not remember your last order - every order stands alone. That sounds limiting, but it is exactly what lets REST apps handle huge traffic and scale easily.

Slide 24 - HTTP methods & mappings

ON SCREEN

A table mapping GET, POST, PUT, DELETE to their Spring annotations, and a note on today vs Session 2.

SAY (read this aloud)

Let's connect those verbs to actual Spring code. Every interaction with a REST API uses an HTTP verb that expresses your intent, and Spring gives each verb its own annotation that links a web address to one of your Java methods.

GET is for reading - it is 'safe', meaning it never changes anything - and it maps to at-GetMapping. POST is for creating new data - at-PostMapping. PUT is for updating - at-PutMapping. And DELETE is for removing - at-DeleteMapping. Picking the right verb is part of good design - for instance, because a GET never changes data, it is safe to retry it or cache it.

Now, for today's lab, we only use at-GetMapping. Why just reading? Because reading is the simplest possible case - you can test it just by typing a URL into your browser. The other three

verbs - which change data - need a database to be meaningful, so they arrive together in Session 2 when we build full CRUD: Create, Read, Update, Delete. Walk before we run.

Slide 25 - Spring Initializr

ON SCREEN

A table explaining each field on start.spring.io and what we choose: Maven, Java, Group, Artifact, Jar, Java 17, web starter.

SAY (read this aloud)

So how do we actually start a project? You never begin from an empty folder - that would be painful. Instead, there is an official website called Spring Initializr, at start dot spring dot i-o, that generates a complete, ready-to-run project skeleton for you. Many IDEs have it built in too.

Let me walk you through the form, because you will fill it in today. Project - the build tool; we choose Maven. Language - Java. Spring Boot - pick the latest stable version. Group - this is your organisation's package name, reversed; we use com dot tcs dot training. Artifact - the project's name; we use springboot-demo. Packaging - choose Jar, the self-contained executable, not War. Java - version 17, a stable long-term version. And Dependencies - just add Spring Web for now.

Then you click Generate, and it downloads a zip file. That zip is your starting project. You unzip it, open it in your IDE, and you are ready to go. It is like getting a pre-built furniture frame instead of cutting the wood yourself.

Slide 26 - Project structure

ON SCREEN

A folder tree of the generated project, with bullets and a key box about what's missing.

SAY (read this aloud)

Once you unzip the project, you will see this folder layout. It follows a standard convention, so the good news is that once you learn it, every Spring Boot project in the world looks familiar. Let me give you the quick tour.

There is a folder for your application code, which holds your main class. There is a resources folder for configuration and static files - that is where application dot properties lives. There is a separate folder for your tests. There is the pom dot xml, which lists your dependencies. And there is the Maven wrapper, which lets you run Maven even without installing it.

Now here is what I find the most telling thing about this slide - look at what is missing.

EMPHASISE

Point out the absence - it is Spring Boot's value made visible: there is no web dot xml, no tomcat folder, no pile of XML configuration anywhere. The server is embedded and the configuration is automatic. That empty space, that lack of clutter, IS the benefit of Spring Boot, right there in front of them.

Slide 27 - pom.xml anatomy

ON SCREEN

A pom.xml snippet showing parent, dependencies, and the Spring Boot Maven plugin.

SAY (read this aloud)

Let me explain the pom dot xml file, because it is the heart of a Maven project. The name stands for Project Object Model, and its job is to declare what your project needs and how to build it. A Spring Boot pom has three parts worth knowing.

First, the parent. Your project inherits from something called spring-boot-starter-parent. This is the magic source of all those tested, compatible versions - which is why you rarely write a version number yourself. Second, the dependencies section - this is where your starters go; right now, just the web starter. And third, a build plugin - the spring-boot-maven-plugin - which is the tool that packages everything into that single runnable fat JAR we discussed.

So notice how short this file is. The reason it can be so short is that the parent and the starters are quietly doing all the heavy lifting for you. Less for you to write means less for you to get wrong.

Slide 28 - application.properties

ON SCREEN

Examples of common properties - server.port, logging levels - and the idea of overriding conventions.

SAY (read this aloud)

This little file, application dot properties, sits in the resources folder, and it is where you override Spring Boot's defaults when you want to. And remember the philosophy - if you are happy with the defaults, this file can stay almost empty. That is perfectly normal and perfectly fine.

Let me show you the kind of thing you put here. server dot port equals 9090 changes the port away from the default 8080. You can set the logging level to control how chatty the logs are. You can set the application's name, database connection details, and so on. Each line is just a simple key equals value.

Here is the mental model I want you to hold. Spring Boot has already made a hundred sensible choices for you. This file is simply where you say, 'thanks, but for this one thing, do it my way instead'. By the way, there is also a YAML version of this file, with a dot y-m-l extension, that organises the same settings in a nested style - you will run into both in real projects, so do not be surprised.

Slide 29 - Common mistakes

ON SCREEN

A list of frequent beginner errors - controller outside the scanned package, missing annotations, port conflicts.

SAY (read this aloud)

Let me save you some pain by pre-warning you about the bugs you are most likely to hit today. If you recognise these, you will fix them in seconds instead of struggling for an hour.

Number one, and the most common - a controller placed outside the main class's package, so component scanning never finds it, and your endpoint returns a 404 with no clear reason. The fix: keep controllers in a sub-package of the main class. Number two - forgetting an annotation like `@RestController` or `@Service`, so Spring never registers the bean. Number three - a port conflict: 'port 8080 already in use', usually because a previous run is still going; just stop it, or change the port. And number four - expecting an injected field to work in a class that Spring did not create; injection only happens for Spring-managed beans.

So if something just refuses to work today, run down this list first. Nine times out of ten, your answer is right here.

DELIVERY TIP

If you have a real war story - 'I once lost an hour to the package-scan 404' - tell it here. A personal anecdote makes this stick far better than the bullet list, and it reassures them that even experienced developers hit these.

Lab, Recap & Close

Slide 30 - Lab preview

ON SCREEN

A preview of what they'll build in the hands-on lab: a `HelloController` and a `GreetingService`.

SAY (read this aloud)

Alright, time to look ahead to the fun part - the lab. In a moment you will create your very first Spring Boot project from the Initializr, and then build two small classes that mirror the layers we discussed.

First, a `GreetingService`, marked with `@Service` - this holds the business logic; its job is simply to return a greeting message. Then a `HelloController`, marked with `@RestController` - this exposes a web endpoint and calls the service to get that message. You will inject the service into the controller, run the main method, and then open your browser and watch your greeting come back from your very own web service.

It is deliberately small - we are not building a giant app today. The whole point is to see the entire machine work, end to end: a request comes in, the controller asks the service, and a response goes back out. Once you see that flow with your own eyes, everything we discussed today clicks into place.

Slide 31 - Recap

ON SCREEN

A recap checklist of the session's key takeaways.

SAY (read this aloud)

Let's pull everything together. Take a breath and look at how much you have learned in one session - this is a lot.

Spring Boot is a layer on top of Spring that removes setup pain through four gifts: auto-configuration, an embedded server, starter dependencies, and production-ready features. Its guiding philosophy is convention over configuration - sensible defaults you only override when needed. A request flows through the front desk, the DispatcherServlet, then to a controller, a service, a repository, and the database - each with one job, which we call separation of concerns. The `@SpringBootApplication` annotation bundles three others, and its `run` method boots the whole app. Dependency injection means the framework creates and wires your objects, giving you flexible, testable code. And REST models things as web addresses acted on by verbs.

If those ideas feel familiar now, you are exactly where you should be. And anything still fuzzy will become solid the moment you type it in the lab.

Slide 32 - Coming up in Session 2

ON SCREEN

A preview of Session 2: building a full REST web service with CRUD.

SAY (read this aloud)

A quick peek at what is next. Today, our app simply ran and returned a greeting. In Session 2, we make it genuinely useful - we turn it into a full REST web service that exchanges real data with any client.

We will build complete CRUD - Create, Read, Update, and Delete - using all four HTTP verbs. We will see how Java objects turn into JSON automatically. We will work with data coming in from the URL and from the request body. And we will test it all with professional tools like Postman and Swagger. The theory will be lighter next time, because the real work of Session 2 is the hands-on lab, where you build the whole API yourself. Something to look forward to.

Slide 33 - Closing

ON SCREEN

The closing blue slide.

SAY (read this aloud)

And that is our foundation laid. You can now explain what Spring Boot does and why it exists - and in just a few minutes, you will have a running web service of your own to prove it.

So let's open the lab sheet and build your very first Spring Boot application together. Stay with me,

follow each step, and do not worry if something breaks - that is part of learning, and we have that handy troubleshooting list. Thank you - and let's take any quick questions before we start coding.

DELIVERY TIP

Common questions to expect: 'do I need to learn core Spring before Boot?' - no, Boot teaches it as you go. And 'why not just always use defaults?' - you mostly do; the properties file is only for the exceptions. Then move straight into the lab while energy is high.